Architetture dei Sistemi di Elaborazione
02GOLYG

Computer Architectures
02LSEYG

Laboratory
0x01

Expected submission:
SURNAME_NAME_MATRICOLA_lab_01.zip

Including:
- program_1 and program_2
- lab_01.pdf (complete this file and export it as PDF)

Submission deadline: 06/10/2026

Simulating a program with gem5 and visualizing pipeline behavior in ASE Studio

ASE Studio provides an interface for running the RISC-V gem5 simulation flow and visualizing the resulting pipeline execution. set up the simulation environment by following the procedure below.

1. Open a terminal

Open a terminal from the linux-based machine.

2. Download the simulation environment

ASE Studio is included as a Git submodule of the **ase_riscv_gem5_sim** repository. Therefore, do not clone the ASE Studio repository separately.

From the terminal, move to the directory where you want to install the environment and clone the parent repository together with all its submodules:

```
$ git clone --branch main --recurse-submodules https://github.com/cad-polito-it/ase_riscv_gem5_sim.git ↵

$ cd ase_riscv_gem5_sim ↵
```

The **--recurse-submodules** option also downloads ASE Studio inside the parent repository.

3. Install the required tools

If you are using a Labinf machine, gem5 and the RISC-V toolchain are already installed. You only need to install ASE Studio:

```
$ ./utils/installation.sh labinf
```

If you are using your local virtual machine, all the required tools are already installed.

If you are using your local machine, install all the required tools:

```
$ ./utils/installation.sh all-ase
```

The complete installation may take several minutes, depending on your machine and Internet connection.

4. Launch ASE Studio

After downloading the repository, ASE Studio can be launched from the desktop shortcut, if available, or directly from the terminal:

```
$ ./ase-studio.sh
```

5. Create a new program

In ASE Studio, create a new project by clicking **New**.

Assign a project name according to the exercise, for example:

program_1

Then click **Create**.

6. Configure the CPU

Configure the CPU according to the configuration specified in each exercise.

Make sure that the selected CPU parameters match the requirements of the exercise before running the simulation.

7. Run the simulation

ASE Studio compiles the program and runs it using gem5 with the selected CPU configuration.

8. Analyze the pipeline

After the simulation is completed, open the pipeline visualization.

Use it to inspect the execution of the instructions through the different pipeline stages and answer the questions reported in the corresponding exercise.

9. Configure the submission settings

Before preparing the submission, open Settings in ASE Studio.

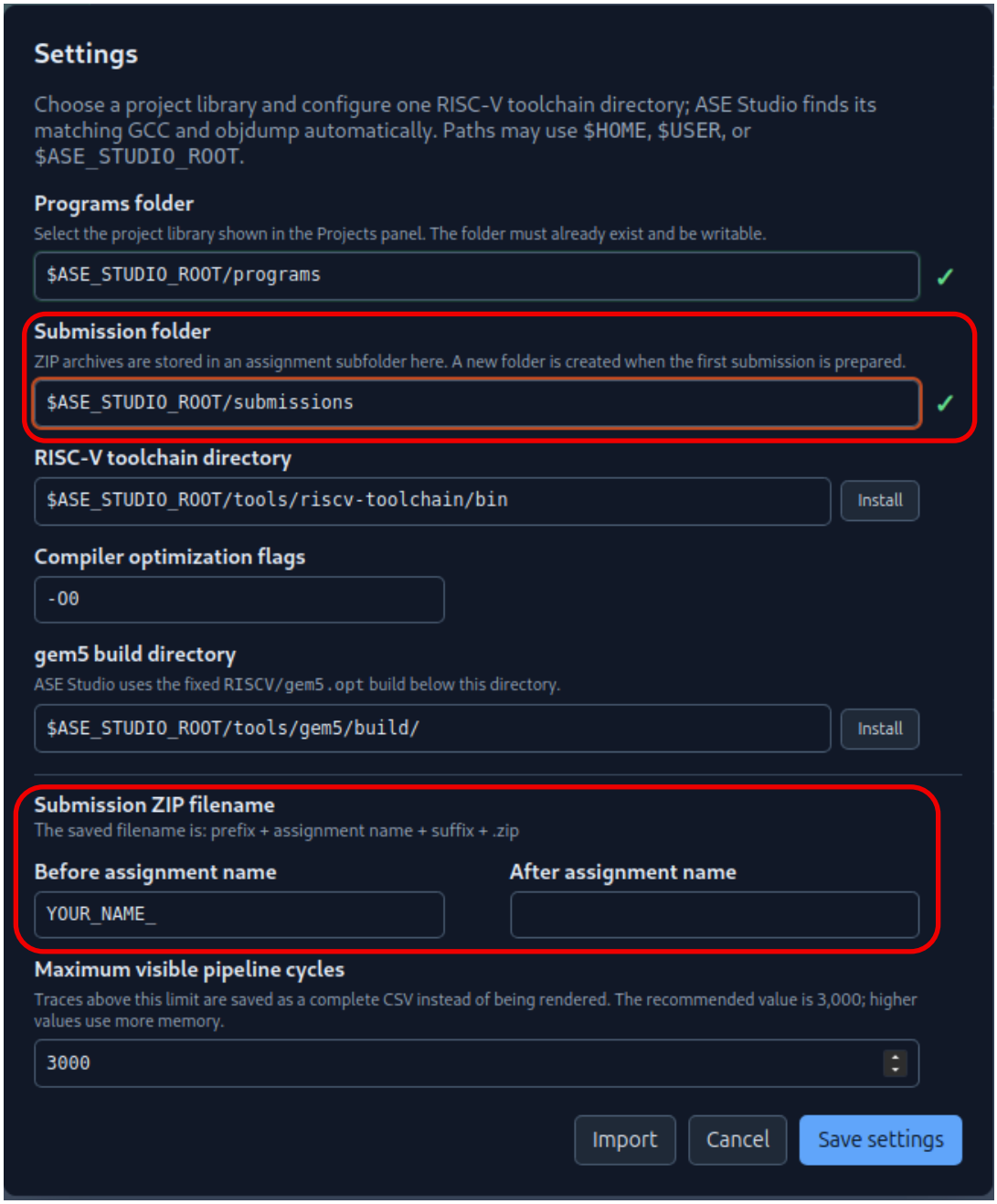
The default submission folder may be changed if needed.

In the **Before assignment name** field, enter:

```
SURNAME_NAME_MATRICOLA_
```

You don’t need to the **After assignment name** field unless explicitly requested.

With this configuration, ASE Studio automatically generates the submission filename using your personal information and the assignment name.



10. Prepare the submission

Complete all exercises and fill in this document with the requested answers and results.

Export the completed document as:

`lab_01.pdf`

Then open Prepare submission in ASE Studio.

Set the Assignment name to:

`lab_01`

Select the required projects:

`program_1`

`program_2`

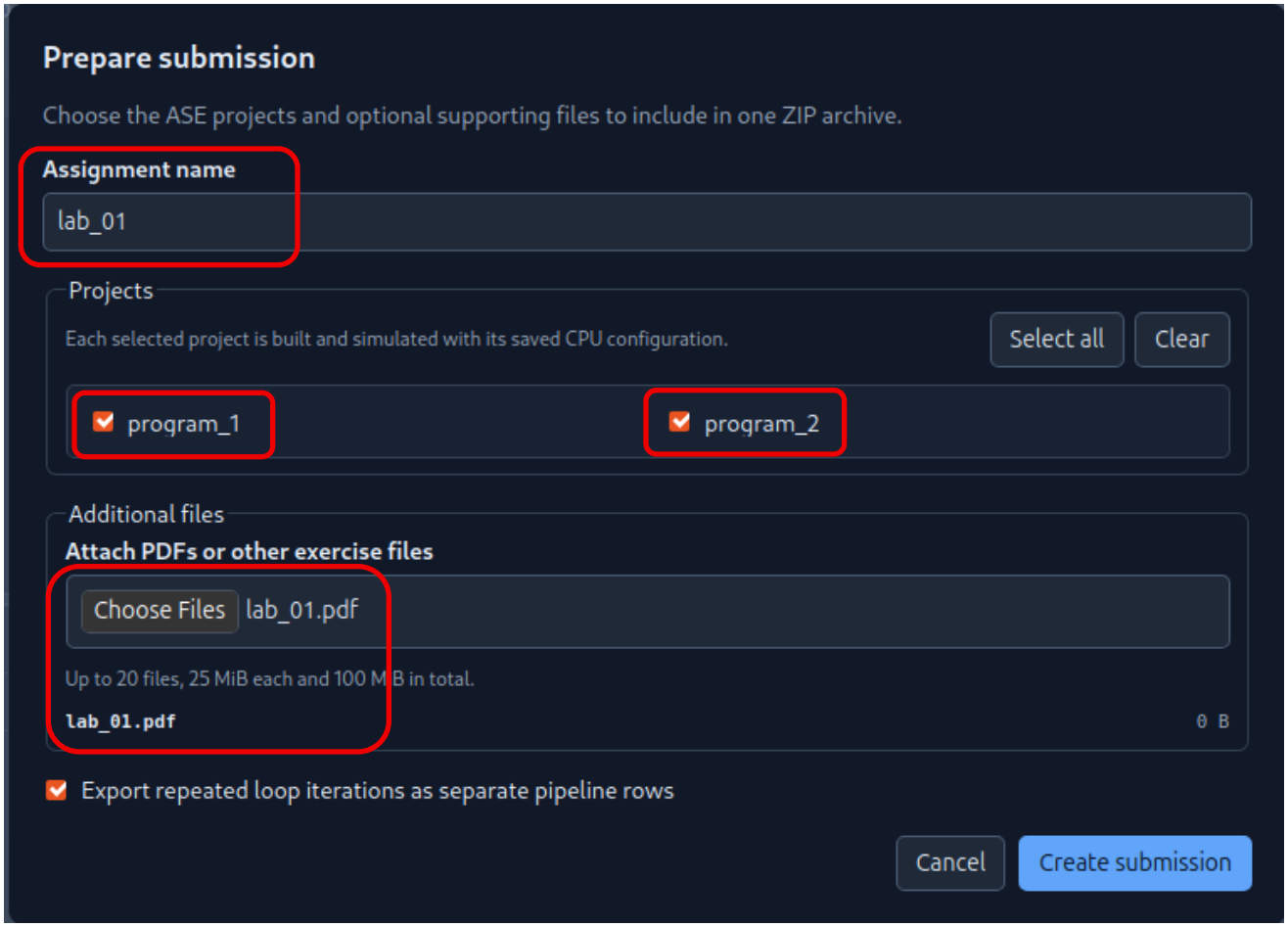
Under Additional files, attach:

`lab_01.pdf`

Finally, click Create submission.

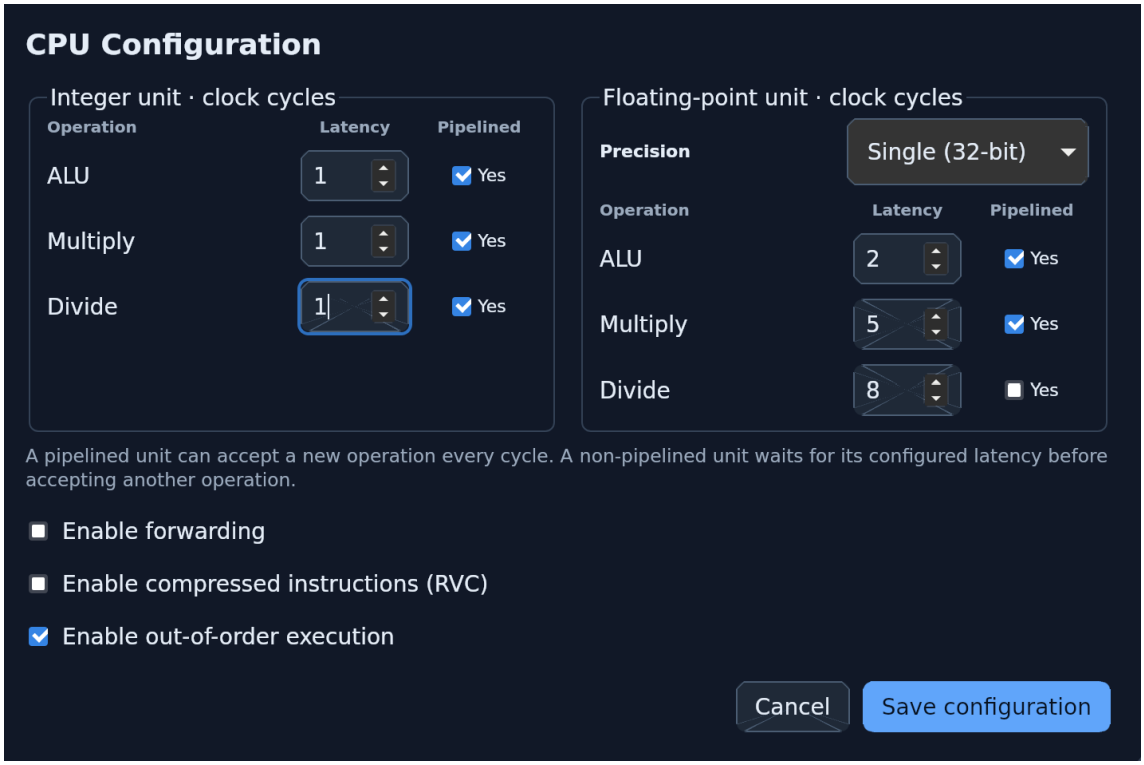
The expected submission file is:

`SURNAME_NAME_MATRICOLA_lab_01.zip`



- CPU Configuration

Please use the following CPU Configuration:



- Exercise 1

Write and run an assembly program that calculates the Fibonacci sequence. Name the project: **program_1**

Use the following template as the starting point for your assembly program:

```
# Data section
.section .data
# Place here your program data. In this example
# two vector of floats, a vector of ints anda single int are defined
T0: .word 0x0BADC0DE

# Code section
.section .text
# The _start label signals the entry point of your program
# DO NOT CHANGE ITS NAME. It must be "_start", not "start",
# not "main", not "start_".
# It's "_start" with a leading underscore and
# all lowercase letters
.globl _start
_start:
Main:
    # Initialize Fibonacci variables
    li x1, 0        # x1 = a = first Fibonacci number (0)
    li x2, 1        # x2 = b = second Fibonacci number (1)
    li x3, 21       # x3 = count = number of terms to generate
    li x4, 0        # x4 = i = loop counter
```

```
    # Loop to generate and print remaining 20 numbers
    addi x4, x4, 1  # i = 1 (start from second iteration)

fib_loop:

    beq x4, x3, End # if i == count, exit loop

    # Calculate next Fibonacci number
    add x5, x1, x2  # x5 = next = a + b

    # Update variables for next iteration
    mv x1, x2       # a = b (previous second becomes first)
    mv x2, x5       # b = next (calculated next becomes second)
    # Increment counter and continue loop
    addi x4, x4, 1  # i++
    j fib_loop      # Jump back to loop start

End:
    # exit() syscall. This is needed to end the simulation gracefully
    li a0, 0
    li a7, 93
    ecall
```

The above assembly code implements the following C code for calculating the Fibonacci sequence:

```c
/* Fibonacci sequence generator in C (pseudocode)
 * Generates the first 21 Fibonacci numbers using iterative approach
 */
int main() {
    int i;          // Loop counter
    int a = 0;          // First Fibonacci number
    int b = 1;          // Second Fibonacci number
    int next;           // Next Fibonacci number
    int count = 21;     // Number of terms to generate

    // Generate and print remaining numbers
    for (i = 1; i < count; i++) {
        next = a + b;   // Calculate next Fibonacci number
        a = b;          // Update: previous second becomes first
        b = next;       // Update: calculated next becomes second
    }

    return 0;
}
```

- Exercise 2

Using the same procedure described above, write and run a RISC-V assembly program. Name the project: **program_2**

The program must:

1. Given two arrays, `v1` and `v2`, each containing 10 signed 8-bit integers, check whether each element of `v1` also appears in `v2` at least once. Store the matching values in a third array, `v3`.

   For example:

```
v1:     .byte     2, 6, -3, 11, 9, 18, -13, 16, 5, 1
v2:     .byte     4, 2, -13, 3, 9, 9, 7, 16, 4, 7
```

   The resulting third array is:

```
v3:     .byte     2, 9, -13, 16
```

2. Set three 8-bit unsigned flags (`flag1`, `flag2`, `flag3`) to indicate following conditions:
   - `flag1` = 1 if `v3` is empty; otherwise, `flag1` = 0.
   - `flag2` = 1 if `v3` is not empty and its elements are in strictly increasing order, i.e. `v3`[i+1] > `v3`[i] for all valid i; otherwise, `flag2` = 0.
   - `flag3` = 1 if `v3` is not empty and its elements are in strictly decreasing order, i.e. `v3`[i+1] < `v3`[i] for all valid i; otherwise, `flag3` = 0.

Then run program_1 and program_2 using the processor configuration specified for this exercise and complete the following table using the simulation results.

Table 1: **Program performance for the specific processor configurations**

Program	Clock cycles	Number of Instructions	Clocks per instruction (CPI)	Instructions per Clock (IPC)
program_1				
program_2				

- Exercise 3

Measure the execution time and CPI of the benchmark programs provided for this laboratory.
The following benchmark programs are included in the laboratory folder:

a) calculate_pi.s

b) insertion_sort.s

For each program, record the number of executed instructions, the number of clock cycles, and the CPI.
Assume a processor frequency of:

1.75 kHz

For the initial scenario, assume that all programs have the same execution weight:

25% for each program.

Then evaluate the following workload scenarios.

- Scenario 1 :
    - program_1 weights 1%
    - program_2 weights 63%
    - calculate_pi weights 25%
    - insertion_sort weights 11%

- Scenario 2:
    - program_1 weights 20%
    - program_2 weights 5%
    - calculate_pi weights 35%
    - insertion_sort weights 40%

- Scenario 3:
    - program_1 weights 20%
    - program_2 weights 31.9%
    - calculate_pi weights 31.4%
    - insertion_sort weights 16.7%

For each scenario, compute the weighted execution time of each program and the total execution time of the workload. Then complete the following table.

Table 2: **Weighted execution time for different workload scenarios**

Program	Initial scenario	Scenario 1	Scenario 2	Scenario 3
`calculate_pi`				
`insertion_sort`				
`program_1`				
`program_2`				
TOTAL Time (@ 1.75kHz)				